



UCBM
UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA



Innovazione e trasformazione digitale

A.A. 2025-26

Modulo 2: Digitalizzazione e ottimizzazione nel settore dell'energia elettrica

Prof. Francesco Conte

Pyomo

Pyomo

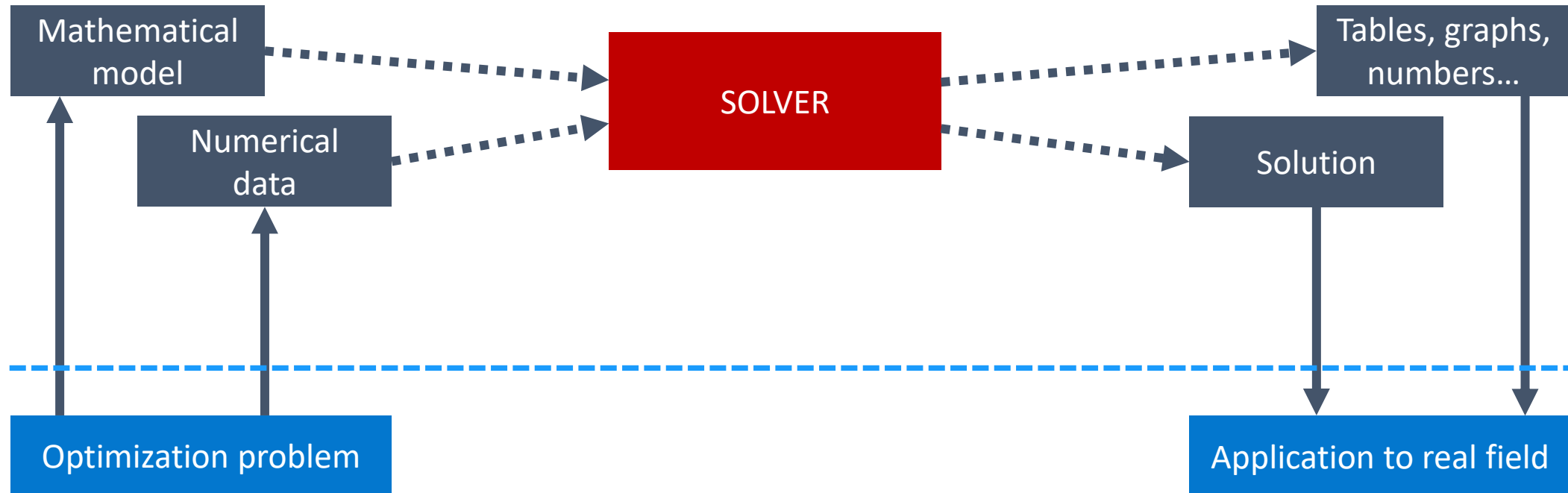


- Pyomo is a Python-based open-source software package that supports a diverse set of optimization capabilities for formulating, solving, and analyzing optimization models.
- Pyomo does not solve problems directly; instead, it calls solvers (such as (e.g., GLPK, CBC, HiGHS, IPOPT, Gurobi and others) to obtain solutions.
- A particular advantage of Pyomo is the closeness between mathematical notation and its syntax in optimization problems. This allows one to be very concise in defining mathematical optimization problems.
- To get documentations and resources, go to: <https://www.pyomo.org/>

Pyomo



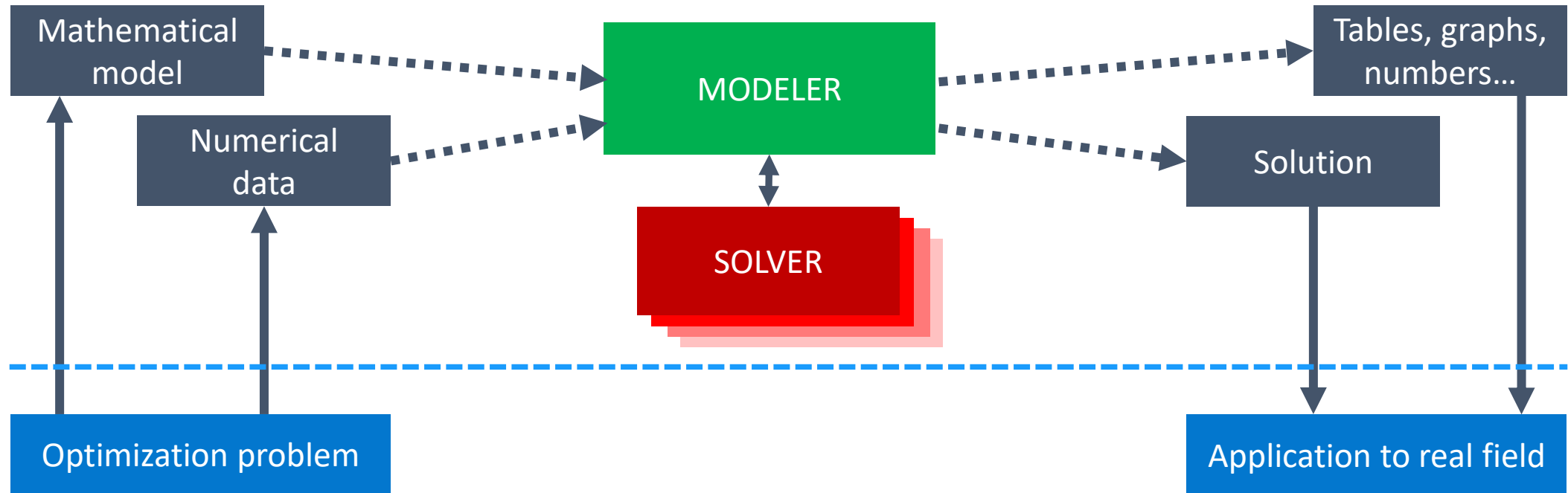
A **solver** is software that receives as input a model describing an optimization problem and outputs the optimal solution of the model.



Pyomo



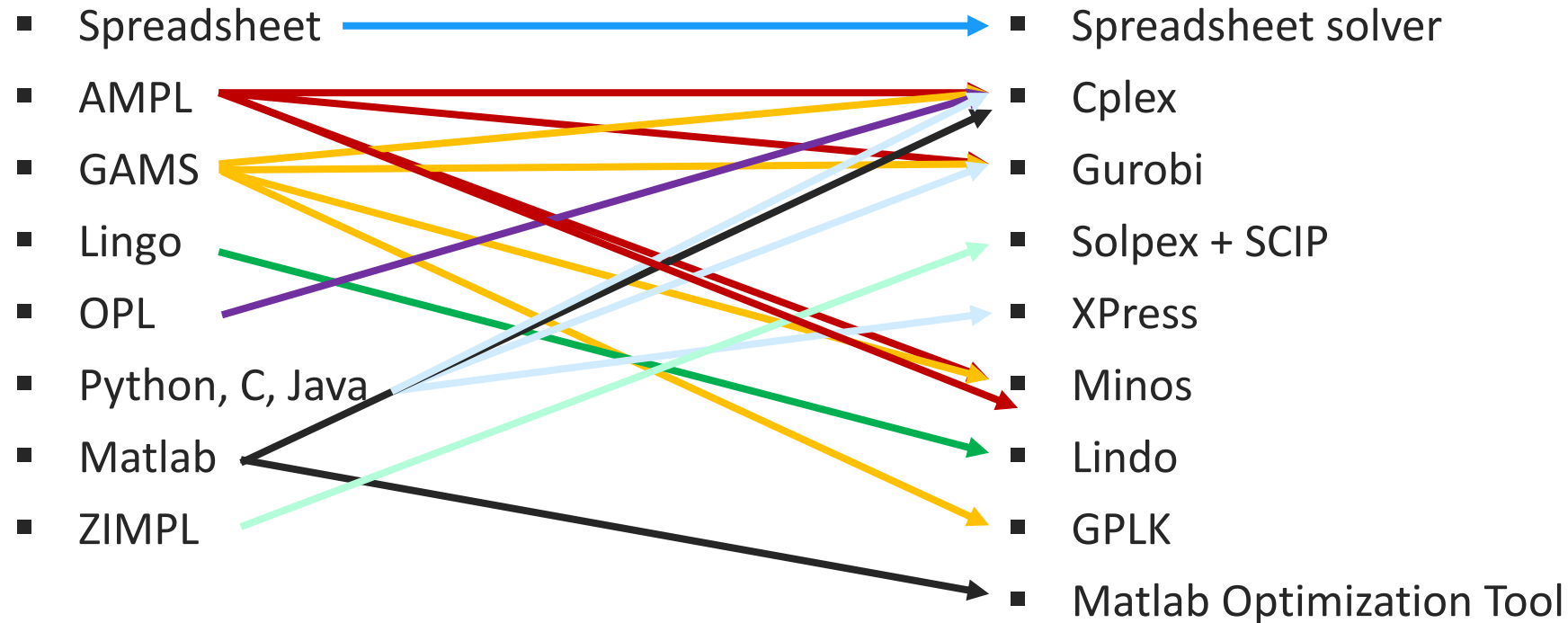
A **modeler** provides an interface between the problem model and the solver.



Pyomo



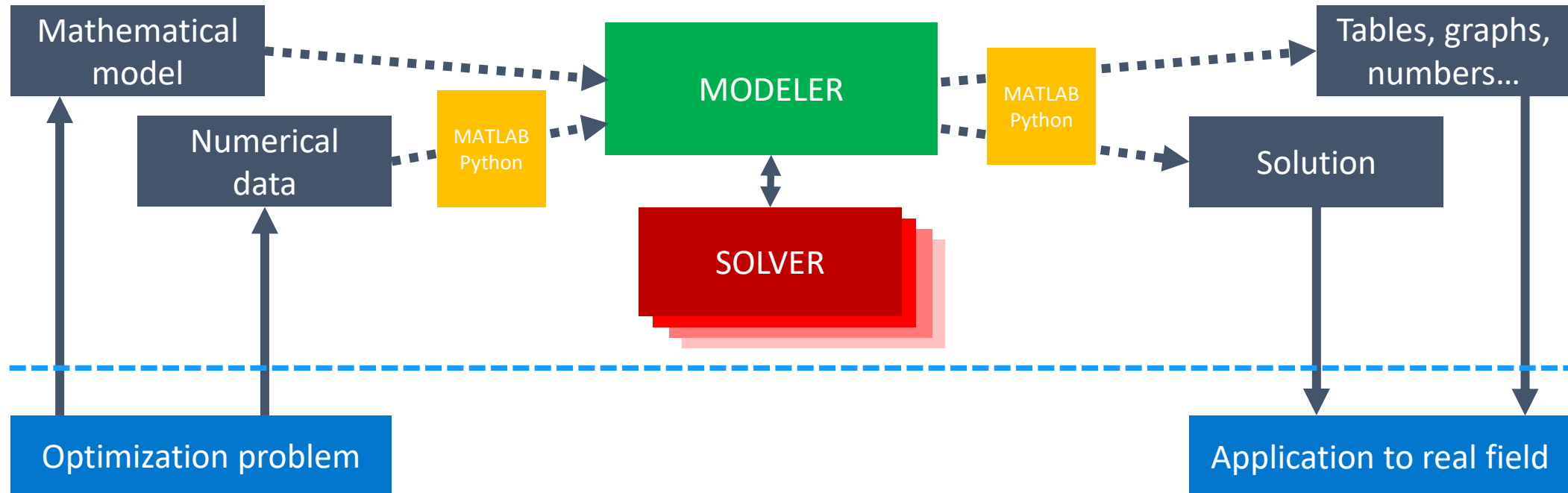
Some possible combinations:



Pyomo



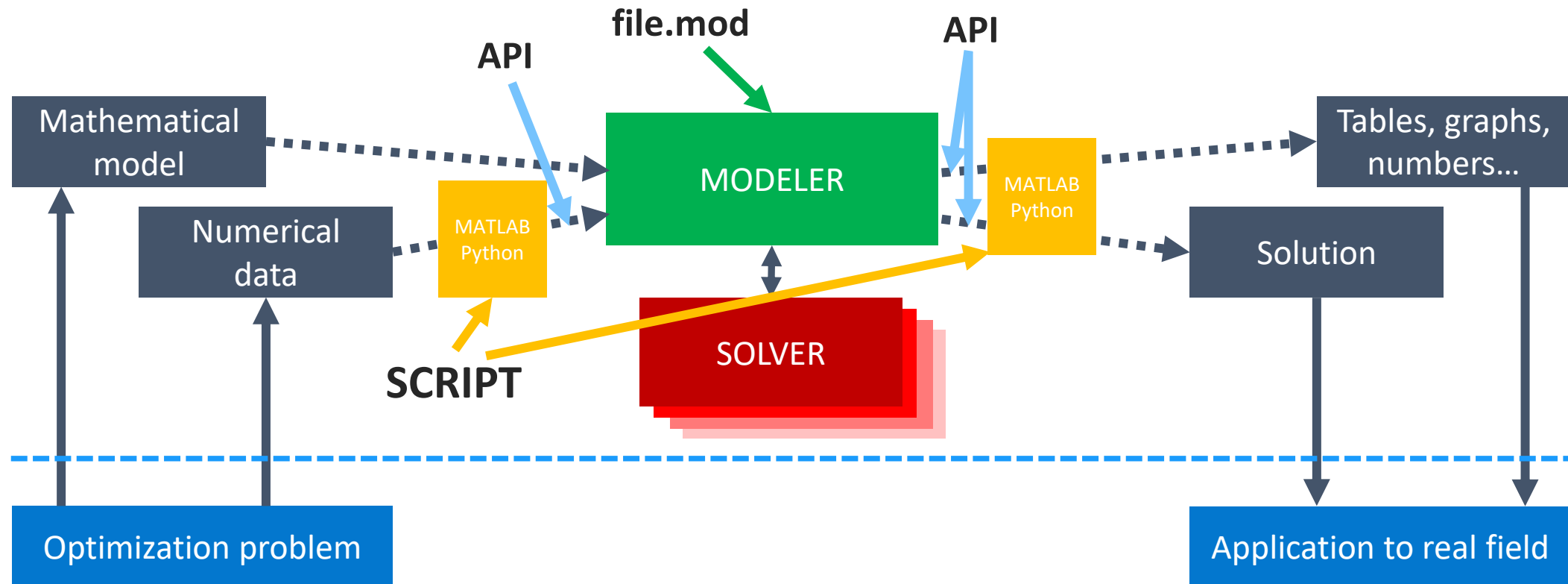
Some modeler can be integrated with general purpose languages. For example, AMPL and GAMS can be interfaced by Python or MATLAB. Pyomo can be interfaced with MATLAB using *pyrun/pyrunfile*



Pyomo



Some modeler can be integrated with general purpose languages. For example, AMPL and GAMS can be interfaced by Python or MATLAB. **HOW IT WORKS?**



In this course we will use Pyomo interfaced with MATALB.



Main commands

- Sets (indices) are declared by the function “Set”
e.g. `model.I = Set(initialize=[10, 3, 6])`
- Sets can be *ordered*. To declare an ordered set we can use the function “RangeSet”
e.g. `model.I = RangeSet(0,n)`
- Declaration of variables is realized with the function “Var”
e.g. `model.x = Var(domain=Reals)`
- Variable can be indexed by sets
e.g. `model.x = Var(model.I, domain=Reals)`
`model.y = Var(model.I, model.J, domain=Reals)`

Pyomo



Main commands

- Variable can be characterized when declared:

```
e.g. model.x = Var(model.I, domain=NonNegativeReals, bounds=(0,10))  
model.x = Var(domain=Binary)
```

- Parameters has not to be declared, they can be scalar or indexed by sets
- Objective function is defined using the function “Objective” referring pre-defined function
e.g.

```
def obj_rule(model):  
    return sum(c[i]*model.x[i] for i in model.I)
```

```
model.obj = Objective(rule=obj_rule, sense=minimize)
```

Pyomo



Main commands

- Constraints are defined using the function “Constraint” referring pre-defined function

e.g.

```
def con1(model, j):  
    return sum(A[j,i]*model.x[i] for i in model.I) <= b[j]
```

```
model.con_1 = Constraint(model.J,rule=con1)
```

Pyomo



Example 0

$$\min_x J = c^T x$$

$$\text{s.t. } Ax \leq b$$

where:

- $x \in \mathbb{R}^n$;
- n is the number of variables;
- $A \in \mathbb{R}^{m \times n}$;
- $b \in \mathbb{R}^m$;
- m is the number of constraints.

FILE example0.py

```
from pyomo.environ import *
from types import SimpleNamespace

def example0_model(parms):

    parms = SimpleNamespace(**parms) #to use struct as in matlab (instead it is a dictionary)
    model = ConcreteModel() #define the pyomo model

    # Sets
    model.I = RangeSet(0, parms.n-1) # variables
    model.J = RangeSet(0, parms.m-1) # constraints

    # Variables
    model.x = Var(model.I, domain=Reals)

    # Define objective function
    def obj_rule(model):
        return sum(parms.c[i]*model.x[i] for i in model.I)
    model.obj = Objective(rule=obj_rule, sense=minimize)

    # Constraints
    def con1(model, j):
        return sum(parms.A[j,i]*model.x[i] for i in model.I) <= parms.b[j]

    # Add constraints to model
    model.con_1 = Constraint(model.J, rule=con1)

    # solve
    results = SolverFactory('gurobi').solve(model)

    out = {"x":value(model.x[:]),
          "solver_status": results.solver.status == SolverStatus.ok,
          "opt_condition": results.solver.termination_condition == 'optimal'
          }

    return out

out = example0_model(parms)
```

Pyomo



Example 0

$$\min_x J = c^T x$$

$$\text{s.t. } Ax \leq b$$

where:

- $x \in \mathbb{R}^n$;
- n is the number of variables;
- $A \in \mathbb{R}^{m \times n}$;
- $b \in \mathbb{R}^m$;
- m is the number of constraints.

FILE example0.py

```
% Problem data
```

```
parms.n = 2; %number of variables
```

```
parms.m = 4; %number of constraints
```

```
parms.A = [-1  1;  
           1 -1;  
          -1  0;  
           0 -1];
```

```
parms.b = [2 3 10 10]';
```

```
parms.c = [1 1]';
```

```
% Call the solution
```

```
pyomo_out = pyrunfile("example0.py", "out", parms=parms);
```

```
% Read solution
```

```
pyomo_out = struct(pyomo_out);
```

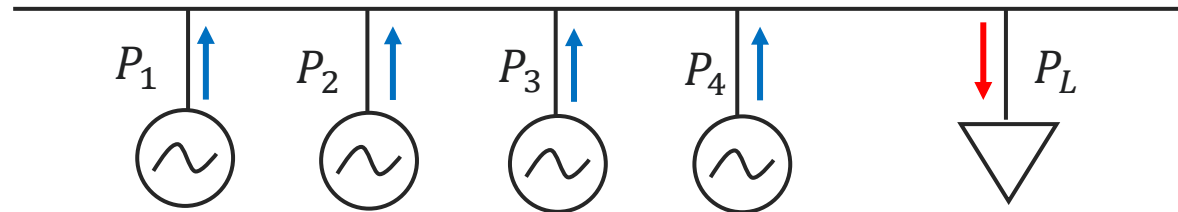
```
x = double(pyomo_out.x);
```

```
disp(['Problem solution is: ', num2str(x)])
```



Example 1

- Dispatching is one of the classical problems in electrical power systems.
- It consists in defining, for a given time interval (15 minutes, 1 hour, 3 hours), which generators have to be in service and, if in service, which are their set-points.
- Dispatching is a typical problem of TSOs, which have the responsibility of always supplying the load demand. Today it is becoming a typical problem also for DSOs and for microgrids.
- The simplest formulation of power dispatch is the one under the **single busbar approximation** and **without the energy market**.





Example 1

The single busbar power dispatch problem

For a given time interval Δt and a set of N generators, let us define

- P_L as the power demand [kW], supposed to be known;
- $c_i(P_i)$ as the generation cost [€/kWh] required to generate P_i kWh from generator G_i , supposed to be known;
- P_i^{max} as the rated power [kW] of generator G_i ;
- P_i^{min} as the technical minimum [kW] of generator G_i .

The problem is defined as follows:

$$\begin{aligned} \{P_i^*, \delta_i^*\}_{i=1}^N &= \arg \min_{\{P_i\}_{i=1}^N} \sum_{i=1}^N c_i(P_i) \Delta t \\ \text{s.t.} \\ \delta_i P_i^{min} &\leq P_i \leq \delta_i P_i^{max} \quad \forall i = 1, 2, \dots, N \\ P_1 + P_2 + \dots + P_N &= P_L \end{aligned}$$

where $P_i, i = 1, 2, \dots, N$ are the active power set-points of generators and δ_i are binary variables that are equal to 1 if generator i is in service, equal to zero if not.



Example 2: the single busbar power dispatch problem

Data

Generator 1:

- $P_1^{max} = 600$ MW
- $P_1^{min} = 150$ MW
- Fuel: coal. Cost: $C_1 = 1.1$ €/MBtu (Mega British thermal unit = 1055.06 J)
- Input-output function: $H_1(P_1) = 510 + 7.2P_1 \Rightarrow c_1 = c_1(P_1) = C_1H_1(P_1) = 510 \cdot 1.1 + 7.2 \cdot 1.1 \cdot P_1 = \mathbf{561 + 7.92P_1} = \mathbf{a_1 + b_1P_1}$

Generator 2:

- $P_2^{max} = 400$ MW
- $P_2^{min} = 100$ MW
- Fuel: oil type A. Cost: $C_2 = 1.0$ €/MBtu
- Input-output function: $H_2(P_2) = 310 + 7.85P_2 \Rightarrow c_2 = c_2(P_2) = C_2H_2(P_2) = 310 \cdot 1 + 7.85 \cdot 1 \cdot P_2 = \mathbf{310 + 7.85P_2} = \mathbf{a_2 + b_2P_2}$

Generator 3:

- $P_3^{max} = 200$ MW
- $P_3^{min} = 50$ MW
- Fuel: oil type B. Cost: $C_3 = 1.05$ €/MBtu
- Input-output function: $H_3(P_3) = 78 + 7.97P_3 \Rightarrow c_3 = c_3(P_3) = C_3H_3(P_3) = 78 \cdot 1.05 + 7.97 \cdot 1.05 \cdot P_3 = \mathbf{81.9 + 8.3685P_3} = \mathbf{a_3 + b_3P_3}$

Total load to be satisfied: $P_L = 500$ MW.



Example 2: the single busbar active power dispatch problem

Problem to be implemented:

$$\begin{aligned}\{P_i^*, \delta_i^*\}_{i=1}^3 &= \arg \min_{\{P_i\}_{i=1}^3} \sum_{i=1}^3 c_i(P_i) \Delta t \\ &= \arg \min_{\{P_i\}_{i=1}^3} \sum_{i=1}^3 (\delta_i a_i + b_i P_i)\end{aligned}$$

s.t.

$$\delta_i P_i^{\min} \leq P_i \leq \delta_i P_i^{\max} \quad \forall i = 1, 2, 3$$

$$P_1 + P_2 + P_3 = P_L$$



FILE example1.py

```
from pyomo.environ import *
from types import SimpleNamespace

def example1_model(parms):

    parms = SimpleNamespace(**parms) #to use struct as in matlab (instead it is a dictionary)

    model = ConcreteModel() #define the pyomo model

    # Set
    model.I = RangeSet(0, parms.N-1) #variable

    # Variables
    model.P = Var(model.I, domain=NonNegativeReals) #powers
    model.d = Var(model.I, domain=Binary) #in-service

    # Define objective function
    def obj_rule(model):
        return sum(model.d[i]*parms.a[i] + parms.b[i]*model.P[i] for i in model.I)

    model.obj = Objective(rule=obj_rule, sense=minimize)

...
```



FILE example1.py

...

```
# Constraints definition
def con1(model, i):
    return model.P[i] <= model.d[i] * parms.Pmax[i]

def con2(model, i):
    return model.P[i] >= model.d[i] * parms.Pmin[i]

def con3(model):
    return sum(model.P[i] for i in model.I) == parms.Pl

# Add constraints to the model
model.con_1 = Constraint(model.I, rule=con1)
model.con_2 = Constraint(model.I, rule=con2)
model.con_3 = Constraint(rule=con3)

# solve
results = SolverFactory('gurobi').solve(model)

out = {"P": value(model.P[:]),
       "d": value(model.d[:]),
       "total_cost": value(model.obj),
       "solver_status": results.solver.status == SolverStatus.ok,
       "opt_condition": results.solver.termination_condition == 'optimal'
       }

return out

out = example1_model(parms)
```



UCBM
UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA



Innovazione e trasformazione digitale

f.conte@unicampus.it